

Edocs — Web Technology Project Report

Enterprise e-contracts and digital document management. This report answers each item of the course instruction (requirements 1–9 and the bonus points) and points to the code and evidence that satisfy it.

Course	Web Technology, 2026–2027 (Faculty of Information Technology)
Project	Edocs — e-contract lifecycle, e-signature and tamper-evident archive
Repository	edocs_frontend (React 19) · edocs_backend (Spring Boot 4.1) · documentation/
Companion document	Edocs_Backend_Implementation_Log.pdf — what was built, how it was verified, known gaps
Date	3 October 2026

Contents

Requirement traceability	3
1. Problem, users, objectives and requirements	4
2. Domain and data modeling	7
3. Responsive web pages	11
4. Architecture and backend	13
5. Relational and non-relational persistence	14
6. Authentication, security and performance	14
7. RabbitMQ messaging	15
8. Role-based access control	15
9. Git workflow and pull request	17
10. Bonus points	17

Requirement traceability

Each row is one item of the instruction, where it is answered in this report, and the artifact that demonstrates it.

#	Instruction item	Where answered	Evidence in the repository
1	Problem, users, objectives, scope, requirements, quality attributes, user stories	Section 1	This report; project document (.docx)
2a	ER-style / UML class diagram	Section 2.1	edocs_class_diagram_v4.png, diagrams/er_diagram.png
2b	Abstract domain concepts	Section 2.2	diagrams/domain_concepts.png
2c	Technology-oriented schema (PK, FK, constraints, indexes)	Section 2.3	db/migration/V1__init_schema.sql, Mongo @Document classes
3	Responsive web pages	Section 3	edocs_frontend/, screenshots/ (desktop, tablet, mobile)
4	Architecture style + backend framework	Section 4	edocs_backend/ (Spring Boot, layered modular monolith)
5	Relational + non-relational databases	Section 5	PostgreSQL via JPA/Flyway, MongoDB via Spring Data
6	Secure authentication incl. OAuth2; performance	Section 6	security/, identity/AuthService, CacheConfig, application.yml
7	RabbitMQ for email, SMS and other events	Section 7	messaging/, diagrams/messaging.png
8	RBAC authorization model	Section 8	security/RolePermissions, @PreAuthorize, DocumentAccess
9	Git usage and pull request	Section 9	Commit plan and PR text
B1	DevOps for deployment	Section 10.1	Dockerfiles, docker-compose.yml, .github/workflows/ci.yml
B2	Testing and QA	Section 10.2	src/test (47 executed tests + 9 container twins), browser E2E
B3	New technology	Section 10.3	Spring Boot 4 / Java 21 virtual threads, envelope crypto, hash chain

1. Problem, users, objectives and requirements

1.1 Problem statement

Contract management is still largely paper-based. Printing, couriering and filing add 3–5 days to a typical turnaround and cost \$20–\$50 per contract. Paper is easily lost or altered, nobody can see where a contract is in its approval chain, versions multiply over email, and proving compliance (eIDAS, ESIGN/UETA, GDPR, SOC 2) from a paper archive is close to impossible. Simple e-signature tools do not manage the contract lifecycle, and document management systems lack legally meaningful signatures. **Edocs** combines both in one secure, auditable web platform.

1.2 Target users

User	Need	Role in Edocs
Legal counsel (law firms, in-house legal)	Draft from vetted templates, redline, route, send for signature	legal
External / internal signers	Review and sign quickly and safely from any device	signer
Administrators (IT, operations)	Manage members, policies, retention and legal holds	admin
Auditors and compliance officers	Read-only trail, verify integrity, export court-ready evidence	auditor
Industries	Real estate, finance, HR, procurement — any high-volume agreement workflow	—

1.3 Objectives

- Cut the time from draft to final signature from days to hours, and make status visible in real time.
- Make every signed contract tamper-evident and provable: content hash, signer evidence and a hash-chained audit trail.
- Enforce least privilege per role and strict tenant isolation between organizations.
- Keep confidential contract text encrypted at rest with per-document keys.
- Ship a maintainable, testable and deployable system: one artifact, containerised, with CI.

1.4 Scope

In scope (implemented)	Out of scope (future phases)
Password + MFA sign-in, Google/Microsoft OAuth2 SSO, JWT sessions, logout revocation	SAML / LDAP federation, SCIM provisioning
Templates, document drafting, versions, comments, shares	Real-time co-editing with live cursors (CRDT)
Multi-party sequential e-signature with OTP intent and evidence	Hardware-token QES through a certified trust service provider
Workflows, routing rules, review escalation job	Visual BPMN designer
WORM-style archive, legal holds, hash-chain verification, evidence export	Anchoring to a public blockchain (simulated here)
Email and SMS through RabbitMQ (SMTP + pluggable SMS gateway)	Native mobile apps, push notifications
Dashboard KPIs, risk bands, compliance controls	ML-based clause risk scoring (heuristic used instead)

1.5 Functional / business requirements

ID	Requirement	Implemented by
FR-01	Users sign in with email + password; administrators must pass a one-time code (MFA).	POST /auth/login, /auth/mfa/verify

ID	Requirement	Implemented by
FR-02	Users can sign in with Google or Microsoft (OAuth2 authorization code) if invited.	/oauth2/authorization/{provider}
FR-03	Legal and admins create documents from templates with parties, value and dates.	POST /documents
FR-04	Editors save new versions; history and comments are kept; sealed documents are immutable.	PATCH /documents/{id}, comments
FR-05	Documents are shared with view/comment/sign access and the recipient is emailed.	POST /documents/{id}/shares
FR-06	Sending for signature notifies signers; reminders can be sent per party.	POST /send, /parties/{pid}/remind
FR-07	Signers prove intent with an OTP and sign in order; the last signature seals the document.	POST /otp, /signatures
FR-08	Every action is recorded in a tamper-evident audit trail that can be verified.	audit_logs hash chain, /audit-logs/verify
FR-09	Auditors search the archive, see timelines and export an evidence package.	/archive, /documents/{id}/evidence
FR-10	Admins apply/release legal holds, manage members, roles and workspace policy.	/archive/{id}/legal-hold, /members, /settings
FR-11	Compliance controls are evaluated against live data on demand.	POST /compliance/checks
FR-12	Dashboard shows velocity, pending signatures, compliance posture, volume, risk, bottlenecks.	GET /dashboard

1.6 Measurable quality attributes

Attribute	Target	How it is met / measured
Performance	p95 API latency < 300 ms for list/detail calls at demo scale	Measured 20–70 ms on the seeded workspace (curl timings); caches, indexes, batch fetching, gzip
Scalability	Stateless API instances behind a load balancer	JWT bearer auth, no server session for the API; virtual threads; HikariCP pool
Reliability	No lost or phantom messages on rollback; failed deliveries retried	Publish after commit, 3 retries with backoff, dead-letter queue; optimistic locking
Security	0 high-severity OWASP Top-10 issues in the API surface	BCrypt, lockout after 5 failures, rate limit 20/min/IP, HTML sanitizer, AES-256-GCM, RBAC, security headers
Integrity	Any change to a past audit entry is detected	SHA-256 hash chain; verified by unit test (edit/delete detected) and /audit-logs/verify
Usability	A signer completes signing in ≤ 3 steps on a 390 px wide phone	Open link → request code → sign; responsive layouts captured at 390/820/1440 px
Maintainability	Each module owns its package; automated tests on every push	Modular packages, 47 executed automated tests, CI pipeline, OpenAPI docs
Availability	Health probes for orchestration	/actuator/health/liveness and /readiness, Docker HEALTHCHECK

1.7 User stories and acceptance criteria

US-1 · Send a contract for signature (legal counsel)

As legal counsel, I want to create a contract from a vetted template and send it to the signers, so that signing starts without paper.

- Given I am signed in as legal, when I create a document from the “Mutual NDA” template with one signer, then it is saved as *draft* at workflow step 1.
- When I save an edit containing a `<script>` tag, then the stored content has the script removed and the version number increases.
- When I send it for signature, then its status becomes *out_for_signature*, the workflow moves to step 3 and each signer is emailed through RabbitMQ.
- When I try the same as an auditor or signer, then the API answers 403 “Your role does not allow this action.”

Automated by: AbstractFlowTest.documentLifecycleFromDraftToSealed, rbacIsEnforcedPerRole

US-2 · Sign with proof of intent (signer)

As an external signer, I want to sign the contract on my phone with a one-time code, so that my signature is legally attributable to me.

- Given the contract is out for signature and I am the next signer, when I request a code, then a 6-digit OTP is sent to me and expires in 10 minutes.
- When I submit a wrong code, then signing is refused with 422 and nothing changes.
- When I submit the right code, then my signature is recorded with IP, certificate and evidence hash; if I am the last signer the document becomes *signed* with a SHA-256 seal.
- When the second signer tries to sign before the first, then the API answers 409 “Waiting for ... to sign first.”

Automated by: AbstractFlowTest.documentLifecycleFromDraftToSealed, SigningRulesTest

US-3 · Prove integrity to a court (auditor)

As an auditor, I want to verify that the audit trail has not been altered and export an evidence package, so that the organization can defend a contract in a dispute.

- When I call verify, then every audit entry's hash is recomputed and the result is *intact*.
- If any past entry were edited or deleted, then verification reports the first broken sequence number.
- When I export evidence for a signed contract, then it contains the document hash, signers, signatures, workflow and the audit trail.
- After the last signature, an “anchored” entry appears in the timeline once the RabbitMQ event is consumed.

Automated by: AuditServiceTest, AbstractFlowTest (anchor wait + evidence assertions)

2. Domain and data modeling

2.1 UML class diagram (conceptual)

The conceptual model from the project document defines three bounded areas: identity and access, the core document domain, and operations and audit. It uses generalisation (*Person* → *User*, *ContentItem* → *Document* → *Contract*, *ContentItem* → *Template*), composition (*Organization* ◻ *Membership*, *Contract* ◻ *Party*), aggregation and associations with cardinalities.

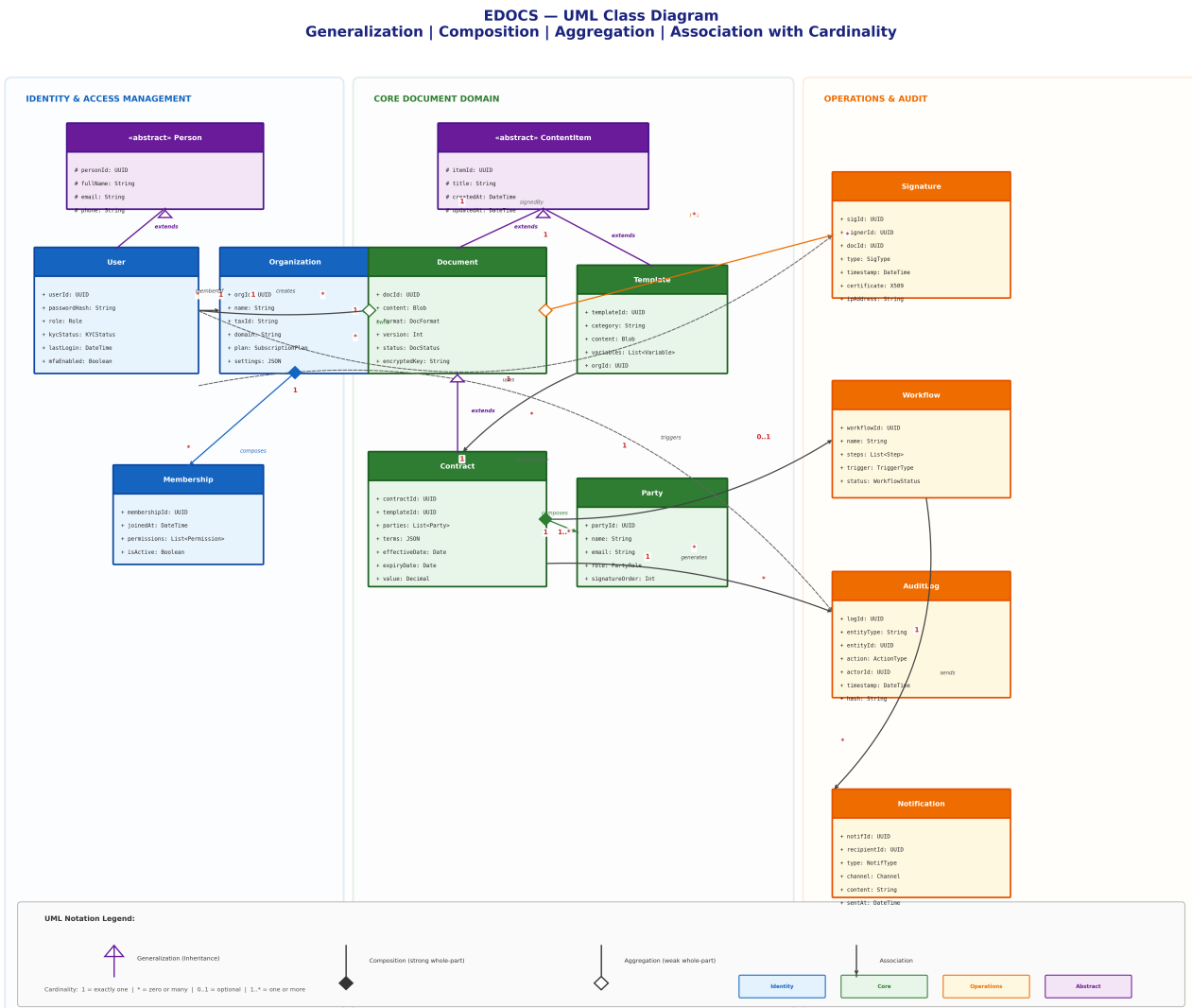


Figure 1 — Edocs UML class diagram (conceptual model)

2.2 Principal domain concepts

Independent of technology, Edocs is about **actors** moving **data objects** through a **contract lifecycle**:

Domain concepts — actors, processes and data objects

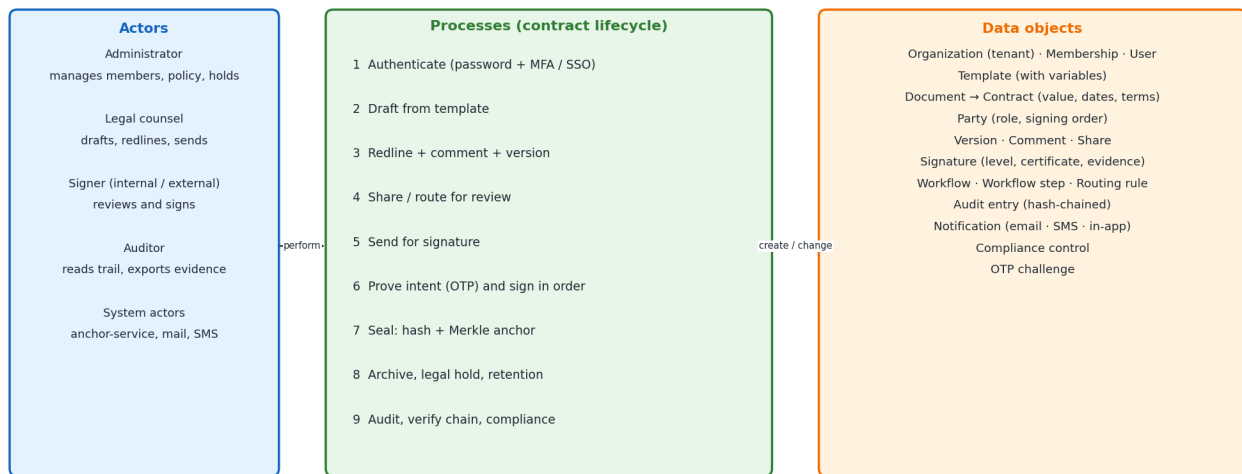


Figure 2 — Actors, processes and data objects

Concept	Meaning
Organization (tenant)	A customer workspace. All data is scoped to it; users join through a Membership.
Contract	A document with parties, value, dates and terms. Moves draft → in review → out for signature → signed → archived.
Party	A person who must sign, approve or view a contract, with a position in the signing order.
Signature	Proof that a party signed: assurance level (SES/AES/QES), time, IP, certificate subject and an evidence hash.
Workflow	The approval path of one contract (draft, legal review, counterparty, QES) with its current step.
Audit entry	An immutable record of who did what, when and from where, linked to the previous entry by hash.
Notification	A message to a person over email, SMS or in-app.
Compliance control	A policy check (eIDAS, GDPR, SOC 2, ISO 27001) evaluated against the live data.

The conceptual model is refitted into a PostgreSQL schema (strong consistency, foreign keys and CHECK constraints for the transactional core) and MongoDB collections (flexible, append-heavy operational data). The schema is versioned with Flyway (V1__init_schema.sql); Hibernate runs with `ddl-auto=validate`, so the application refuses to start if the code and schema drift apart.

PK = primary key FK = foreign key (red) UQ = unique CK = check constraint IX = index



Conceptual	Physical decision	Why
ContentItem → Document → Contract	@MappedSuperclass ContentItem; JOINED inheritance: documents + contracts (PK = FK doc_id)	Contract-only columns stay out of documents; one PK space
Person → User	@MappedSuperclass Person; table users	Shared attributes without an empty table
Organization.settings: JSON	Typed columns signature_level, retention_years, require_2fa with CHECKs	The database can enforce valid policy values
Membership.permissions: List	Derived from the role at runtime (RolePermissions)	One source of truth for RBAC; no drift
Template.variables: List	Child table template_variables (template_id, position)	First normal form, ordered values
Workflow.steps: List	Child table workflow_steps; live status derived from current_step	Steps are queryable and constrained
Document.content: Blob	MongoDB document_contents, AES-256-GCM per document; wrapped key in documents.encrypted_key	Large, versioned, encrypted payload away from OLTP rows
AuditLog, Notification	MongoDB collections with compound indexes; audit uses unique (orgId, seq)	Append-heavy, schema-flexible; seq + hash chain detect tampering

Conceptual	Physical decision	Why
Signature.certificate: X509	certificate TEXT (subject + level + serial); raw capture in Mongo signature_artifacts	Evidence is preserved without bloating the table

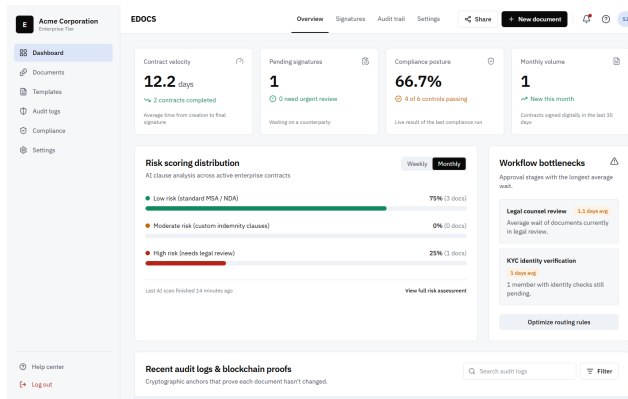
Key constraints and indexes

- **Integrity:** sealed documents must have a SHA-256 (ck_documents_sealed); a legal-hold date requires a hold; expiry $\geq$ effective date; contract value $\geq$ 0; emails stored lower-case.
- **Uniqueness:** one membership per (user, organization); one party per (contract, email); one share per (document, email); one signature per party; one workflow per document; audit (orgId, seq).
- **Referential actions:** signatures use ON DELETE RESTRICT, so signed evidence can never be cascaded away. Shares, workflows and parties cascade with their document.
- **Indexes for the hot paths:** (org_id, updated_at DESC) for document lists, (org_id, status) for dashboards, a partial index for sealed documents, parties(email) and shares(email) for signer visibility, Mongo (recipientId, createdAt DESC) for the notification bell.
- **Concurrency:** documents.row_version provides optimistic locking (HTTP 409 on a lost update).

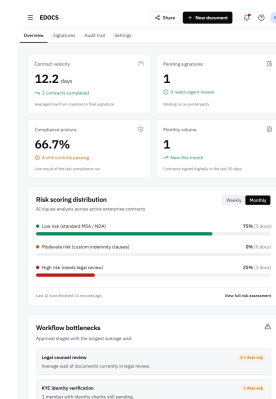
3. Responsive web pages

The frontend is React 19 + TypeScript + Tailwind CSS v4 (edocs_frontend/). It covers sign-in (password, MFA, SSO), the dashboard, document list, smart editor, workflow, signing portal, archive, templates, compliance and settings. The screenshots below were captured from the running app **connected to the real Spring Boot backend** (headless Chrome at 1440, 820 and 390 px). The sidebar collapses into a menu, tables keep their key columns, and actions stay reachable with a thumb on phones.

Desktop 1440 px



Tablet 820 px



Mobile 390 px

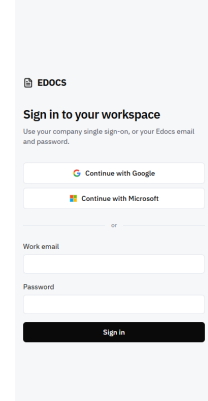
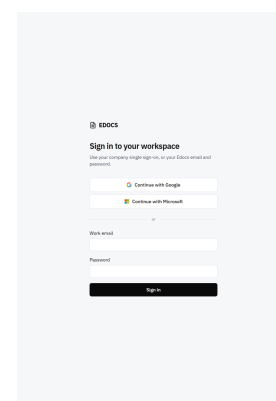
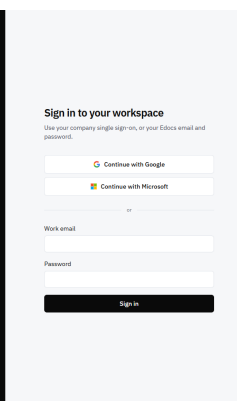
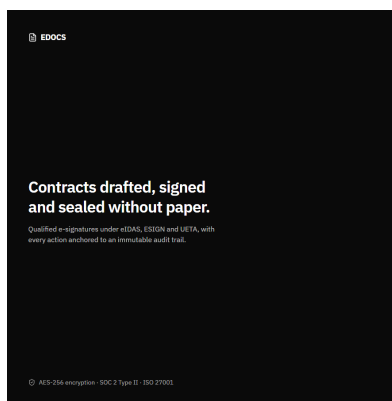
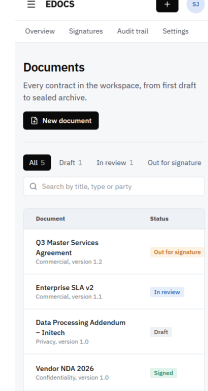
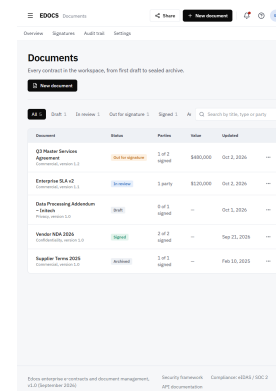
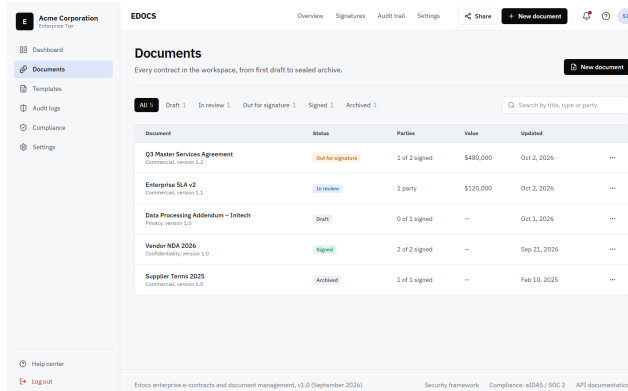
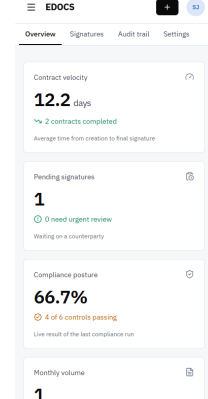


Figure 4 — Dashboard, documents and sign-in at three breakpoints (live backend data)

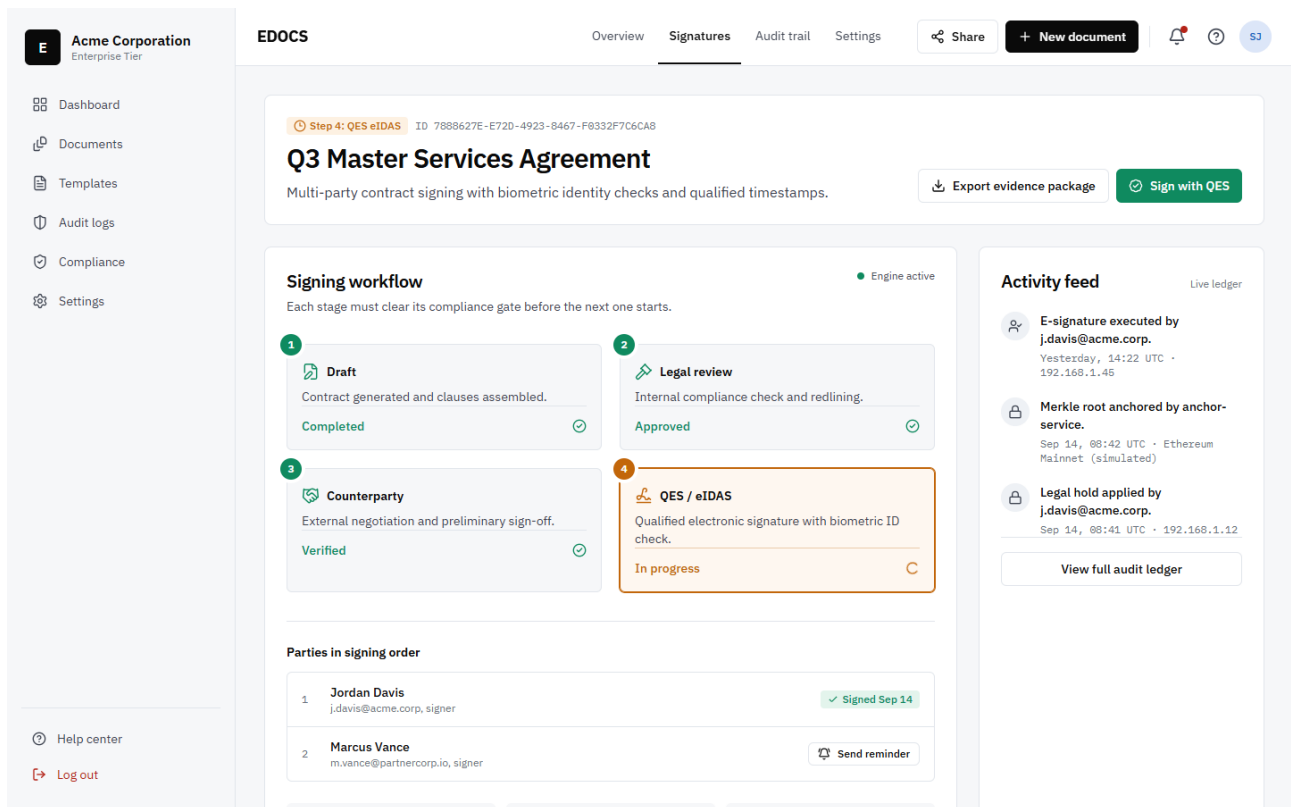


Figure 5 — Signing workflow: steps, parties in order and the hash-chained activity feed

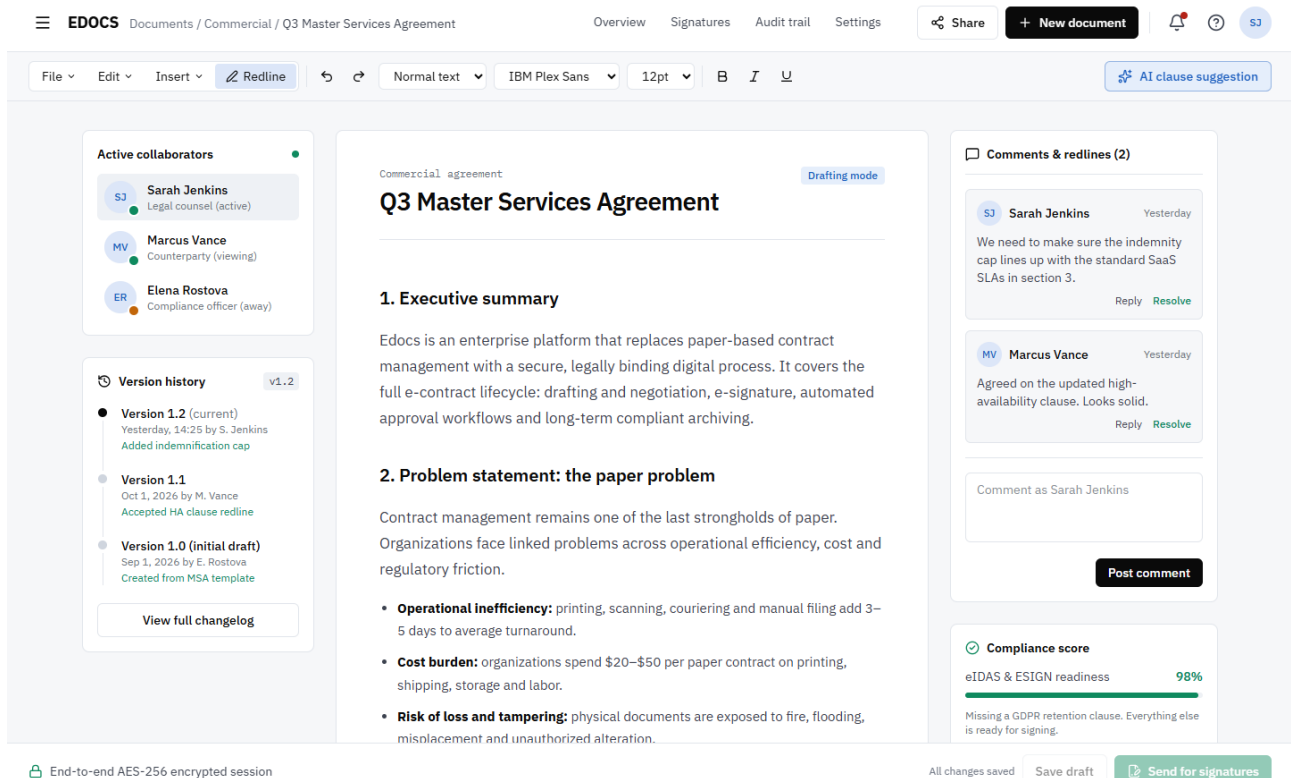


Figure 6 — Smart contract editor with redlines, versions and comments (decrypted from MongoDB)

4. Architecture and backend

Style: a **layered modular monolith with event-driven messaging**, which is phase 1 of the hybrid evolutionary approach recommended in the project document. Each business capability (identity, document, signing, workflow, audit, notification, compliance, dashboard) is its own package with controller → service → repository layers. Side effects (email, SMS, anchoring) leave the request path through RabbitMQ, so they can later be extracted as microservices (Strangler Fig) without changing the core.

Edocs — layered modular monolith with event-driven messaging

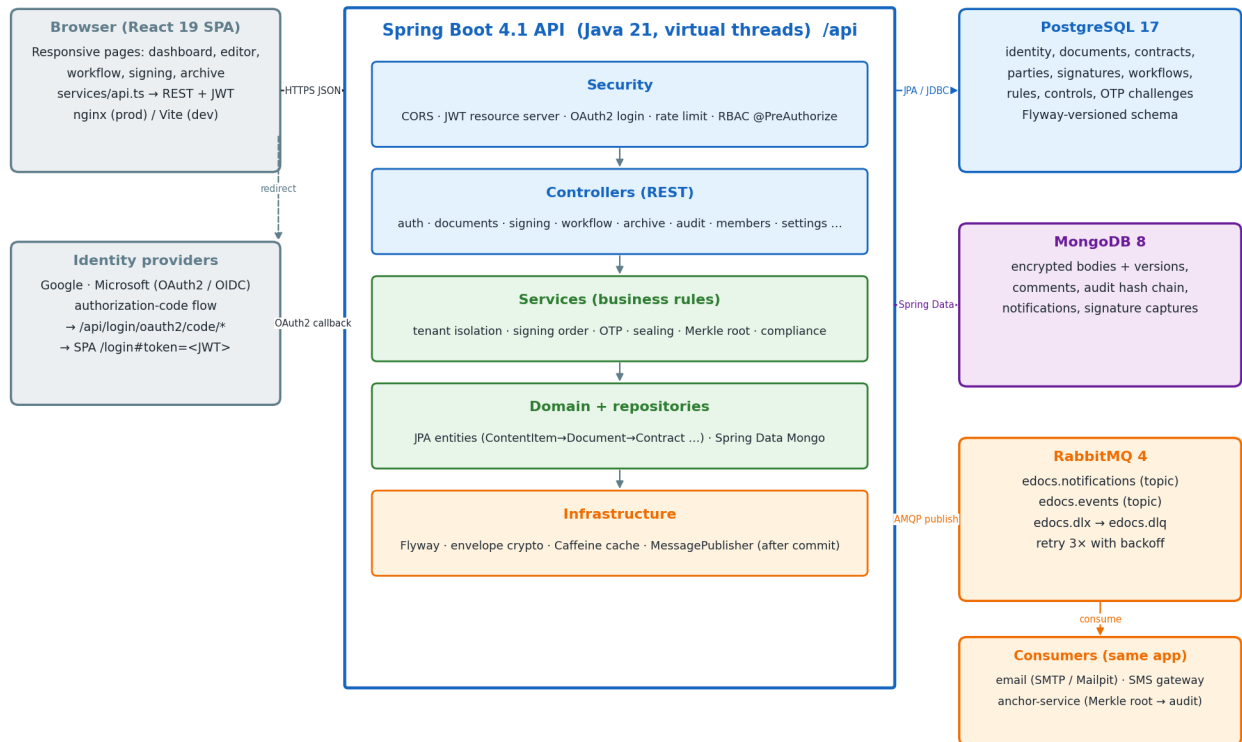


Figure 7 — Deployment and layering

Layer	Responsibility	Examples
Security	Authenticate every request, enforce CORS, RBAC, rate limits	SecurityConfig, JwtAuthUserConverter, LoginRateLimiter
Controllers	HTTP contract identical to the frontend's api.ts; validation	DocumentController, SigningController, AuthController
Services	Business rules and transactions	DocumentService (lifecycle), SigningService (order, OTP, seal)
Domain / repositories	Entities, aggregates, persistence	Contract, Party, AuditLog, Spring Data JPA + Mongo
Infrastructure	Crypto, cache, messaging, migrations	EnvelopeCrypto, CacheConfig, MessagePublisher, Flyway

The frontend switches from mock data to this API by setting

VITE_API_BASE_URL=http://localhost:8080/api. Every endpoint in services/api.ts is served with the same JSON shapes, and interactive OpenAPI documentation is published at /api/docs.

5. Relational and non-relational persistence

Data	Store	Reason
Organizations, users, memberships	PostgreSQL	Identity needs uniqueness, foreign keys and transactions
Documents, contracts, parties, shares, templates	PostgreSQL	Relational queries (signer visibility, dashboards) with constraints
Signatures, workflows, routing rules, compliance, OTP	PostgreSQL	Legal evidence protected by RESTRICT keys and CHECKs
Document bodies, version snapshots, comments	MongoDB	Large, nested, versioned payloads; encrypted blobs; flexible schema
Audit log (hash chain)	MongoDB	High-volume append-only stream; unique (orgId, seq) orders the chain
Notifications, signature captures	MongoDB	Operational data read by recipient; drawn signature images

Both stores are used inside one request where needed. For example, creating a document writes the contract to PostgreSQL and the encrypted body to MongoDB. The evidence package joins signatures (PostgreSQL) with the audit trail (MongoDB).

6. Authentication, security and performance

Mechanism	Implementation
Password sign-in	BCrypt (cost 10); constant-time path for unknown emails; logout for 15 min after 5 failures; 20 attempts/min per IP
Multi-factor	6-digit one-time code, stored only as a salted SHA-256, 10-minute expiry, 5 attempts, delivered by email via RabbitMQ
OAuth2 / OIDC	Spring Security OAuth2 Client with Google and Microsoft (authorization-code flow). Only invited members may sign in; the SPA receives the Edocs JWT in the URL fragment
Session tokens	OAuth2 Resource Server validating HS256 JWTs (issuer, expiry, revocation). Role and active status are re-read per request (30 s cache), so deactivation is immediate
Logout	Token id (jti) added to a deny-list until it expires
Intent to sign	Per-document OTP (SMS when a phone is on file, otherwise email) plus a strict signing order
Data protection	Envelope encryption: AES-256-GCM data key per document, wrapped by a master key; tampering is detected by the GCM tag
Input / output	Bean Validation on every request, OWASP HTML sanitizer for editor content, uniform JSON errors without stack traces
Transport / browser	Strict CORS allow-list, X-Frame-Options DENY, no-referrer, nosniff, no-store cache headers

Performance optimizations

- Caffeine caches (principals 30 s, dashboard 30 s, templates and settings 10 min, archive metrics 60 s) with eviction on writes.
- List endpoints return summaries without bodies; JPA batch fetching (size 50) avoids N+1 queries; open-in-view disabled.
- Indexes chosen for every hot query (Section 2.3); paginated audit log.
- Java 21 virtual threads for request handling; gzip compression; HikariCP pool tuning.
- Slow side effects (SMTP, SMS, anchoring) run asynchronously through RabbitMQ after the transaction commits.

7. RabbitMQ messaging

RabbitMQ topology — email, SMS and domain events

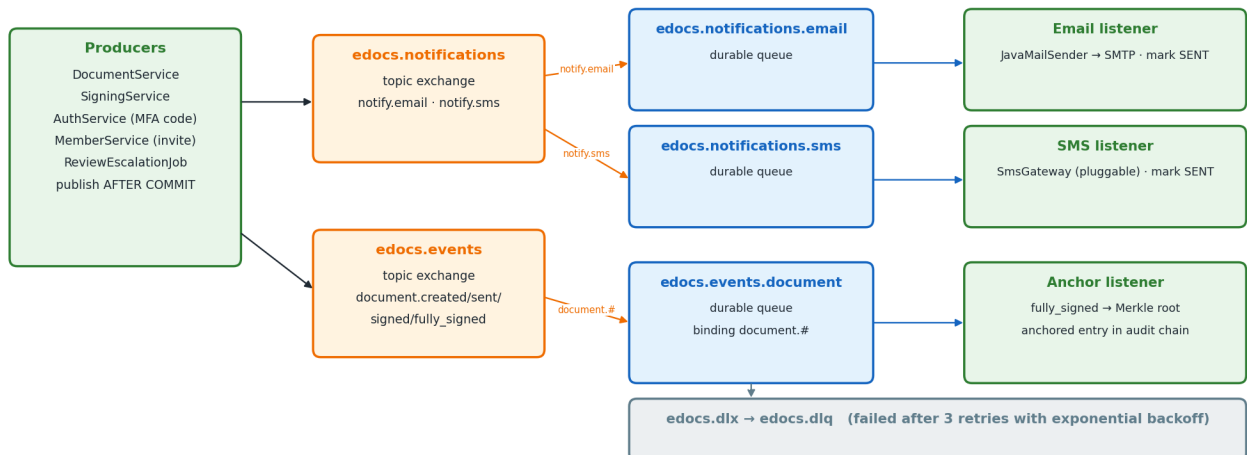


Figure 8 — Exchanges, queues, consumers and dead-lettering

Event / message	Routing key	Consumer effect
MFA sign-in code, invitation with temporary password	notify.email	Email via SMTP (Mailpit in development)
Signing OTP to a member with a phone	notify.sms	SMS via the pluggable SmsGateway
Signature request, reminder, share, fully-signed, escalation	notify.email	Email to party; delivery status written back to MongoDB
document.created / sent / signed	document.#	Domain events for integrations and analytics
document.fully_signed	document.#	Anchor listener appends “Merkle root anchored” to the audit chain

Messages are published **after** the database transaction commits (TransactionSynchronization), so a rolled-back request never sends an email. Consumers retry three times with exponential backoff before a message is dead-lettered to `edocs.dlq`.

8. Role-based access control

Permission	admin	legal	signer	auditor
document:create	Yes	Yes	—	—
document:edit	Yes	Yes	—	—
document:sign	Yes	Yes	Yes	—
document:share	Yes	Yes	—	—
document:delete	Yes	—	—	—
workflow:manage	Yes	Yes	—	—
audit:read	Yes	Yes	—	Yes
audit:export	Yes	—	—	Yes
compliance:run	Yes	—	—	—
template:manage	Yes	Yes	—	—
members:manage	Yes	—	—	—
settings:manage	Yes	—	—	—

Enforcement happens in three layers:

- **Endpoint:** `@PreAuthorize("hasAuthority('document:create')")` on each protected operation; the JWT converter grants `ROLE_x` plus one authority per permission.
- **Resource:** every query is scoped to the caller's organization; signers only see documents they are a party to or that were shared with them (others return 404, so they cannot probe).
- **Business rule:** only the next pending signer may sign; sealed documents cannot be edited or deleted; admins cannot remove their own admin access.
- The same matrix drives the UI (`rbac.ts`); a unit test fails if the frontend and backend matrices ever differ.

9. Git workflow and pull request

The repository uses short-lived feature branches merged into main through pull requests. The frontend lives on frontend; the backend is committed on a backend branch in small, meaningful commits, then a PR to main runs CI.

```
git checkout -b backend
git add edocs_backend/pom.xml edocs_backend/mvnw* edocs_backend/.mvn edocs_backend/.gitignore
git commit -m "build(backend): scaffold Spring Boot 4.1 project with Maven wrapper"
... (one commit per module, see the implementation log for the full list)
git push -u origin backend
gh pr create --base main --head backend --title "Spring Boot backend for Edocs"
```

The full suggested commit sequence (11 commits) and the PR description are listed in the implementation log, Section 7.

10. Bonus points

10.1 DevOps

- **Containers:** multi-stage Dockerfile for the API (layered Spring Boot jar, non-root user, health check, ZGC) and for the SPA (Vite build → nginx with caching and security headers).
- **One-command stack:** `docker compose up --build` starts PostgreSQL 17, MongoDB 8, RabbitMQ 4 (management UI), Mailpit, the API and the frontend, with health-gated start-up order.
- **CI/CD (GitHub Actions):** on every push/PR the backend is built and tested (including Testcontainers journeys on real databases) and the frontend is type-checked and built. On main, both images are pushed to GitHub Container Registry tagged with the commit SHA.
- **Operations:** Actuator liveness/readiness probes, Prometheus metrics endpoint, graceful shutdown, 12-factor configuration through environment variables.

10.2 Software testing and quality assurance

Suite	Type	Tests	Result
RolePermissionsTest	Unit: RBAC matrix, parity with frontend rbac.ts	9	pass
AuditServiceTest	Unit: hash chain, tamper detection, concurrent append retry	5	pass
EnvelopeCryptoTest	Unit: AES-GCM round trip, tamper and wrong-key rejection	5	pass
HashingAndSanitizerTest	Unit: SHA-256, Merkle root, XSS stripping	4	pass
SigningRulesTest	Unit: signing order, outsiders, sealed documents	7	pass
ComplianceEvaluationTest	Unit: control evaluation, fail-closed	5	pass
ArchiveSearchTermsTest	Unit: search parsing	3	pass
EmbeddedFlowTest	Integration: 9 end-to-end journeys through HTTP + security	9	pass
ContainersFlowTest	Integration: same 9 journeys on Docker PostgreSQL/Mongo/RabbitMQ	9	runs in CI (no Docker locally)
Browser E2E (Playwright + Chrome)	System: real SPA against real API, 3 viewports	121 API calls	all 2xx, 0 console errors

JaCoCo measures **81.2% line coverage** (1,439 of 1,772 lines) on `mvn verify`; the CI uploads test and coverage reports as artifacts.

10.3 New technology explored

- **Spring Boot 4.1 / Spring Framework 7 / Hibernate 7 / Jackson 3 on Java 21 virtual threads.**

- **Envelope encryption** (per-document AES-256-GCM data keys wrapped by a master key), the pattern used by cloud KMS services.
- **Tamper-evident audit**: a SHA-256 hash chain plus Merkle roots over each contract's trail (blockchain-style anchoring, simulated).
- **Testcontainers 2** for realistic integration tests, and a Docker-free fallback using embedded PostgreSQL, a MongoDB-wire server and the Apache Qpid AMQP broker.